

# Homework7 参考答案

把 a.txt 映射到两个长度 100K 的 char 数组：sbuf 用共享方式 (MAP\_SHARED)，pbuf 用私有方式 (MAP\_PRIVATE)。page 大小 4KB。100K = 102400 = 25 个 page。a.txt 初始内容全是字符 '0'，共享改动会立刻 sync 到文件。执行的循环：

```
for (idx = 0; idx < ARRAY_LEN; idx++) {
    sbuf[idx] = sbuf[idx] + 1;
    pbuf[idx] = pbuf[idx] + 1;
}
```

即对每个下标先改 sbuf 再改 pbuf，覆盖全部 100K 字节（25 个 page）。

## Problem 1

```
#include <sys/mman.h>
#include <fcntl.h>
#include <unistd.h>
#define LEN (100 * 1024)

int    fd    = open("a.txt", O_RDWR);
char *sbuf = mmap(NULL, LEN, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
char *pbuf = mmap(NULL, LEN, PROT_READ | PROT_WRITE, MAP_PRIVATE, fd, 0);
/* 用完后 close(fd); 不影响已建立的映射 */
```

## Problem 2

mmap 是按需调页，且页表项初始不在内存。每个 page 第一次访问才缺页装入。

sbuf（共享、可写）：每个 page 第一次访问触发一次缺页，把页缓存中的该页以可写方式映射进来；之后的写直接写该共享页，不发生 COW。25 个 page 共 25 次 page fault。

pbuf（私有、可写）：每个 page 第一次“读”触发一次缺页，把文件页以只读方式映射（与文件页缓存共享）；随后第一次“写”触发一次保护故障，执行 COW 复制出私有页。所以每个 page 有 2 次 page fault，其中 1 次是 COW。25 个 page 共 50 次 page fault，其中 25 次是 COW。

合计：page fault 共 25 + 50 = 75 次，其中 25 次执行 COW。

## Problem 3

关键在于循环对每个 idx 先改 sbuf（共享，立刻 sync 到文件/页缓存）再改 pbuf，而 pbuf 私有页在第一次写之前与文件页缓存共享。

idx = 0：先 `sbuf[0]=sbuf[0]+1`，把文件偏移 0 改成 '1'（已 sync）。再读 `pbuf[0]`：此时 pbuf 的 page 0 尚未 COW，缺页装入的就是刚被 sbuf 改过的页缓存页，读到的是 '1'；写入触发 COW，复制出私有页后把偏

移 0 置为 '2'。COW 复制发生在此刻，复制的 页里偏移 1..4095 仍是 '0'（sbuf 还没改到它们）。

idx = 1：sbuf[1] 把文件偏移 1 改成 '1'。再读 pbuf[1]：pbuf 的 page 0 已是私有 副本，读到的是 COW 时刻保存下来的 '0'，加 1 得 '1'，与文件后续改动无关。

其余位置同理：每个 page 的第 0 个字节会读到 sbuf 已同步的 '1' 从而变成 '2'，其余字节读到私有副本里的 '0' 从而变成 '1'。

```
sbuf[0] = '1'    (ASCII 49)
sbuf[1] = '1'    (ASCII 49)
pbuf[0] = '2'    (ASCII 50)  -- 读到了 sbuf[0] 已 sync 的 '1', 再 +1
pbuf[1] = '1'    (ASCII 49)  -- page 0 已 COW, 读到私有副本里的 '0', 再 +1
```